

How to Get Started

This is a short walk through the `pyrulearn` package for inductive rule learning: load your own data from a CSV, fit a native rule learner, inspect the rules it found in a few different formats, and compare two learners across a handful of the benchmark datasets. See the [README](#) for the fuller picture – every learner, interface and demo.

1 Load data from a CSV

`titanic.csv` next to this notebook is the classic Titanic dataset – one row per passenger, `survived` the target. A plain look at it first, the way you'd look at any CSV:

```
[1]: import pandas as pd

df = pd.read_csv("titanic.csv")
df.head()
```

```
[1]:   pclass  survived   sex   age  sibsp  parch   fare embarked
0      1   survived  female 29.0000    0     0  211.3375         S
1      1   survived   male  0.9167    1     2  151.5500         S
2      1     died  female  2.0000    1     2  151.5500         S
3      1     died   male 30.0000    1     2  151.5500         S
4      1     died  female 25.0000    1     2  151.5500         S
```

`pyrulearn.data.io.read_csv` reads the same file straight into a `BooleanDataRepresentation`: it guesses each column's type (numeric vs. nominal), discretizes the numeric ones, and expands everything into binary features – each with its explicit negation – ready to fit on.

Numeric columns are split into `max_intervals` intervals (supervised, decision-tree-based thresholds): more intervals give finer-grained thresholds (more binary features, slower, more prone to overfitting on small data), fewer give coarser ones. The default is 8; we use 3 here.

```
[2]: from pyrulearn.data.io import read_csv

data = read_csv("titanic.csv", target="survived", max_intervals=3)
print(data.spec.n_features,
      "binary features from",
      len(df.columns) - 1,
      "raw columns")
```

52 binary features from 7 raw columns

2 Fit a rule learner

pyrulearn features several native re-implementations of well-known rule-learning algorithms, as well as interfaces to their original implementations. They all can be called on a uniform data representation (internally it is three different ones – Boolean, sparse, and NLists), and return different types of rule-based models (rule sets, lists, etc.) in a uniform representation.

Pypper is a version of the classic RIPPER algorithm. Every learner fits the same way: `fit(data)`.

```
[3]: from pyrulearn.learners.seco import Pypper
```

```
model = Pypper().fit(data)
print(model)
```

```
survived(X) :- pclass(X, V1), V1 \= 3, sex(X, female). % (233/17)
survived(X) :- sex(X, female), fare(X, V1), V1 >= 15.2, embarked(X, V2), V2 \=
S. % (96/7)
% default
died(X) :- true. % (809/500)
```

3 Inspect the rules

The rules (and the whole model) render in a few formats via `to_string(fmt=...)` – useful for different purposes, from reading them as logic to a compact positional encoding. (`ascii=True` sticks to plain ASCII connectives – `to_string()`'s default uses unicode connectives instead.)

```
[4]: # readable logic: "if ... then ..."
print(model.to_string(fmt="logic", ascii=True))
```

```
if pclass != 3 AND sex = female -> survived % (233/17)
elif sex = female AND fare >= 15.2 AND embarked != S -> survived % (96/7)
else -> died % (809/500)
```

```
[5]: # bare conditions, no connective or head
print(model.to_string(fmt="conditions", ascii=True))
```

```
survived: pclass != 3, sex = female % (233/17)
survived: sex = female, fare >= 15.2, embarked != S % (96/7)
% default: died % (809/500)
```

```
[6]: # positional 1/0 pattern over all binary features -- one token per
# feature, 1 where the rule has that condition
print(model.to_string(fmt="pattern"))
```

```
survived: 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
survived: 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
% default: died % (809/500)
```

Small aggregate stats – rule count, length, coverage:

```
[7]: from pyrulearn.evaluation import summarize

summarize(model, data)
```

```
[7]: {'n_rules': 2,
      'mean_length': 2.5,
      'median_length': 2.5,
      'rules_per_target': {'survived': 2},
      'mean_n_covered': 176.5}
```

4 Compare two learners across a few datasets

pyrulearn can access about a hundred small, medium, and large benchmark datasets through its Catalog; run_cv handles the cross-validation, binarization per fold, timing and failure-handling for you. Here's Pypper against PFossil (another native separate-and-conquer learner) on three small datasets:

run_cv discretizes each fold the same way, via its own max_intervals=.

```
[8]: from pyrulearn.experiments.catalog import Catalog
from pyrulearn.experiments.runner import run_cv
from pyrulearn.learners.seco import PFossil

datasets = Catalog.default().select(names=["vote", "tic-tac-toe", "hepatitis"])
results = run_cv([Pypper(), PFossil()], datasets, n_folds=3, max_intervals=3,
↳ verbose=False)
results.groupby(["dataset", "learner"])[["accuracy", "n_rules", "fit_time"]].
↳ mean().round(3)
```

```
[8]:
```

		accuracy	n_rules	fit_time
dataset	learner			
hepatitis	PFossil	0.807	12.000	0.038
	Pypper	0.723	2.000	0.034
tic-tac-toe	PFossil	0.956	15.333	0.124
	Pypper	0.983	8.000	0.252
vote	PFossil	0.945	7.333	0.025
	Pypper	0.952	2.333	0.857

5 Where to go from here

- README.md – a detailed description of the package, with a summary of all provided functionalities, including a list of all native and interfaced learners.
- demos/ – demos of different functionalities. Each can be called in a quick mode (default), which demonstrates its base functionality, and a full mode, which does a more elaborate investigation of an issue. Sample outputs are enclosed in the checked-in reports.